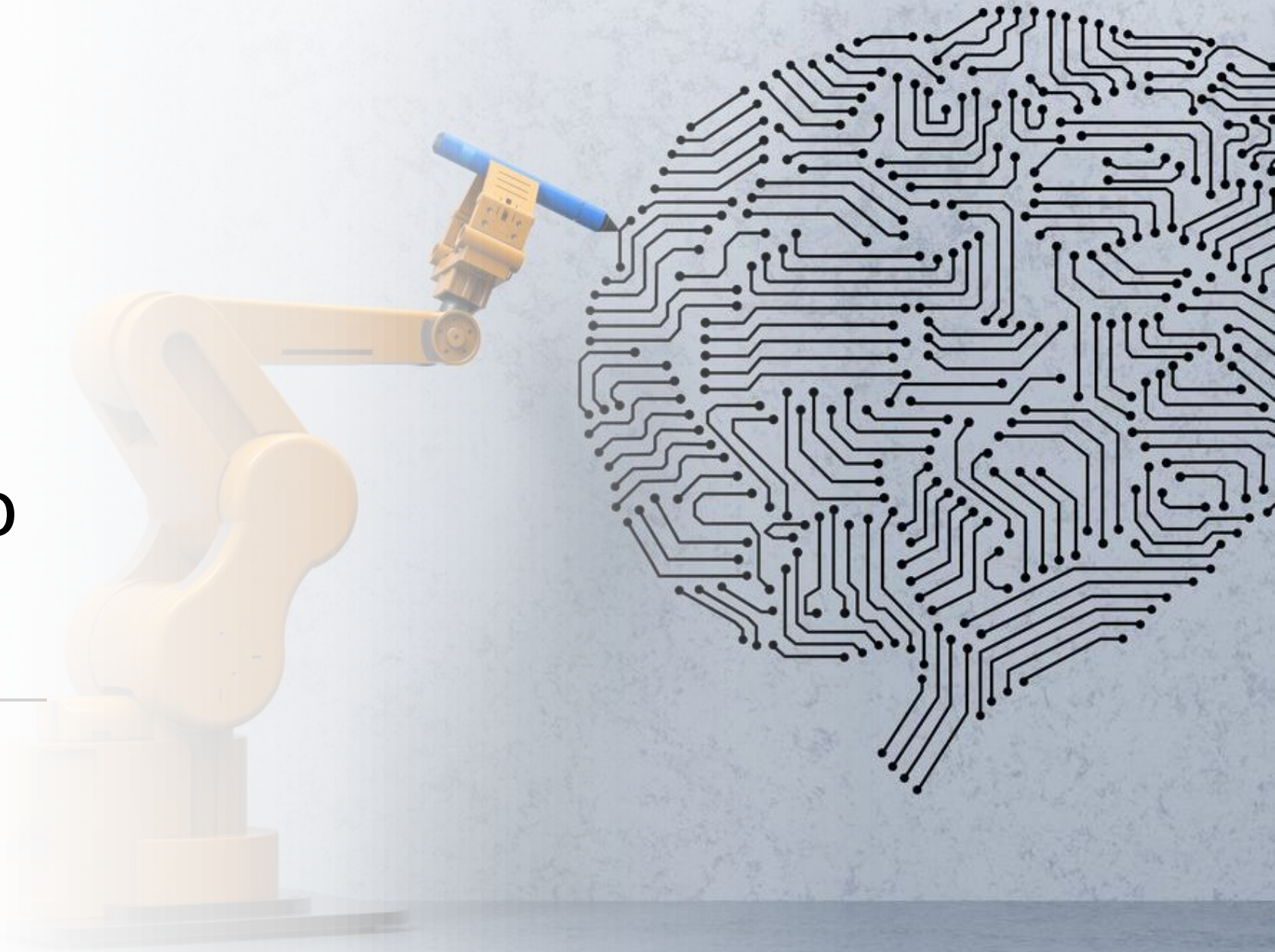




Reconocimiento de Patrones

Clase 9

Clasificación de patrones por medio de funciones de distancia

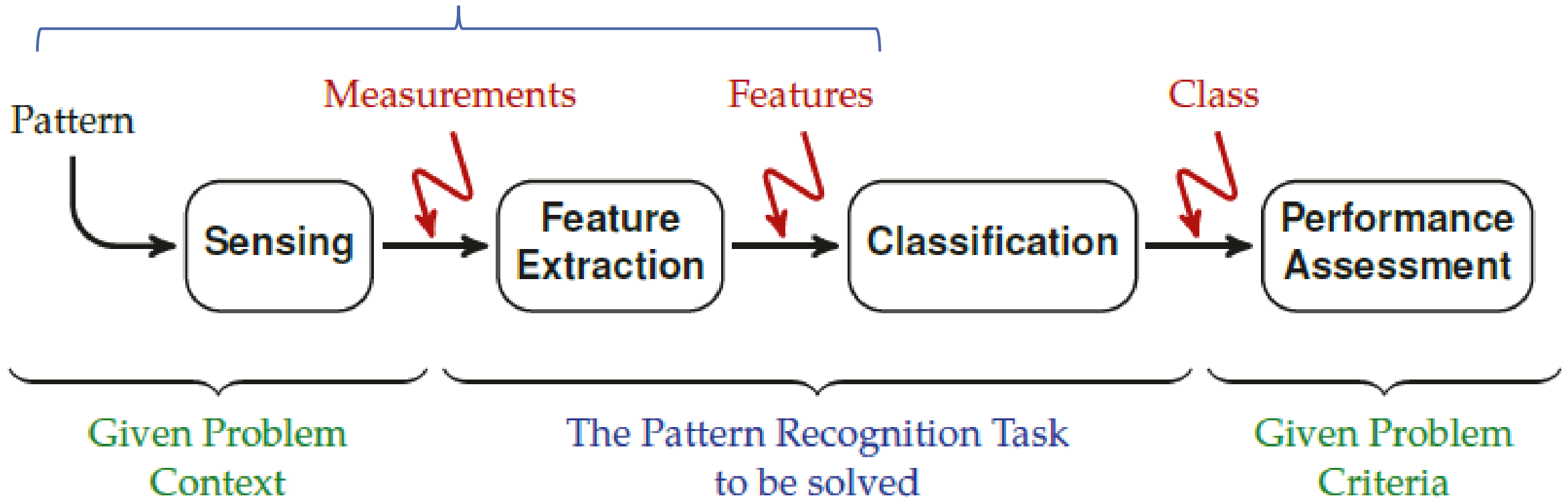


Antes de empezar

- ¿Dudas de la clase anterior?



Hasta ahora



Recordando

¿Qué es la clasificación de patrones?

Proceso de asignar un objeto o vector de características a una de varias categorías predefinidas, basándose en la medición de su semejanza o diferencia con prototipos representativos de cada clase.

Patrón

Vector de características que describe un objeto o fenómeno

Clase / Categoría

Conjunto de patrones con propiedades similares o mismo origen

Función de Distancia

Medida cuantitativa que indica qué tan alejado está un punto de otro en el espacio de características

Clasificación por distancia mínima

- Principio fundamental: Un patrón desconocido x se asigna a la clase cuyo prototipo (centroide) es el más cercano, según una función de distancia.

Distancia Euclidiana

$$d(x, m_k) = \sqrt{\sum_i (x_i - m_{ki})^2}$$

Más común. Mide la distancia geométrica directa entre dos puntos.

Distancia Manhattan

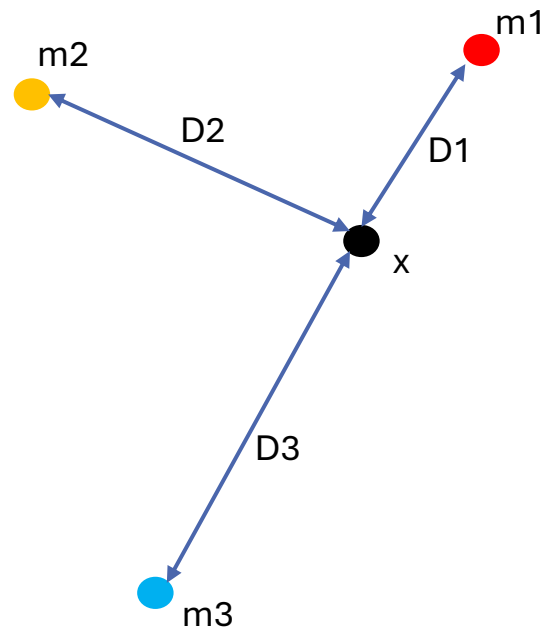
$$d(x, m_k) = \sum_i |x_i - m_{ki}|$$

Suma de diferencias absolutas. Útil en espacios con dimensiones independientes.

Distancia Mahalanobis

$$d^2(x, m_k) = (x - m_k)^T \Sigma^{-1} (x - m_k)$$

Considera la correlación entre variables usando la matriz de covarianza Σ .



- **Un solo prototipo**

La forma más sencilla es calcular la distancia entre un ejemplo y un prototipo de clase:

$$D_i = d(x, m_k), \quad \text{donde } k = 1, 2, \dots, M$$

Es decir, hay M clases y el nuevo ejemplo se asigna a la clase que presente menor distancia.

- **Múltiples prototipos**

Se tiene más de un prototipo por clase tal que la clase w_i tiene N_i prototipos $z_i^1, z_i^2, \dots, z_i^{N_i}$. La distancia entre un patrón arbitrario x y la clase w_i se calcula como:

$$D_i = \min_{\lambda} \|x - z_i^{\lambda}\|, \quad \lambda = 1, 2, \dots, N_i$$

Donde D_i es la mínima distancia entre x y cada prototipo de w_i .

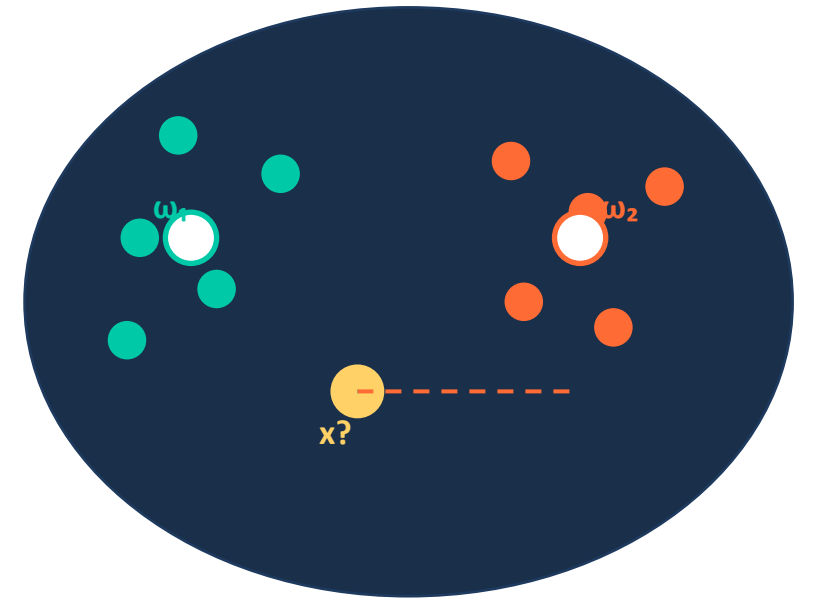
Regla de Decisión por Distancia Mínima

Regla de asignación

Asignar x a la clase ω_k si:

$$d(x, m_k) < d(x, m_j) \quad \forall j \neq k$$

donde m_k es el centroide (prototipo) de la clase ω_k



Ventajas y limitaciones

- | | |
|---|--|
| ✓ Simple de implementar y computacionalmente eficiente | ✗ Sensible a la escala de las variables (requiere normalización) |
| ✓ Efectivo cuando las clases son compactas y bien separadas | ✗ No considera la distribución interna de la clase |

Algoritmos para el Agrupamiento de Datos

Clustering (Agrupamiento): Técnica de aprendizaje no supervisado que organiza datos en grupos (clusters) de manera que los objetos dentro de un mismo grupo son más similares entre sí que con objetos de otros grupos.



Aprendizaje NO Supervisado

- No se dispone de etiquetas de clase a priori
- El algoritmo descubre la estructura intrínseca
- El número de grupos puede ser desconocido
- Evalúa cohesión intra-cluster y separación inter-cluster



Aplicaciones Principales

- Compresión de datos (cuantización vectorial)
- Segmentación de imágenes y señales
- Clasificación de documentos y textos
- Detección de anomalías y patrones ocultos

Medidas de Similitud

Similitud Coseno

$$s(x,y) = (x \cdot y) / (\|x\| \|y\|)$$

Rango: $[-1, 1]$

Mide el ángulo entre vectores, independiente de la magnitud. Muy usada en texto.

Coef. de Correlación de Pearson

$$r = \Sigma (x_i - \bar{x})(y_i - \bar{y}) / [\sigma_x \sigma_y n]$$

Rango: $[-1, 1]$

Mide la relación lineal entre dos variables. $r=1$ perfecta correlación positiva.

Similitud de Jaccard

$$J(A,B) = |A \cap B| / |A \cup B|$$

Rango: $[0, 1]$

Para conjuntos binarios. Útil en datos categóricos o documentos.

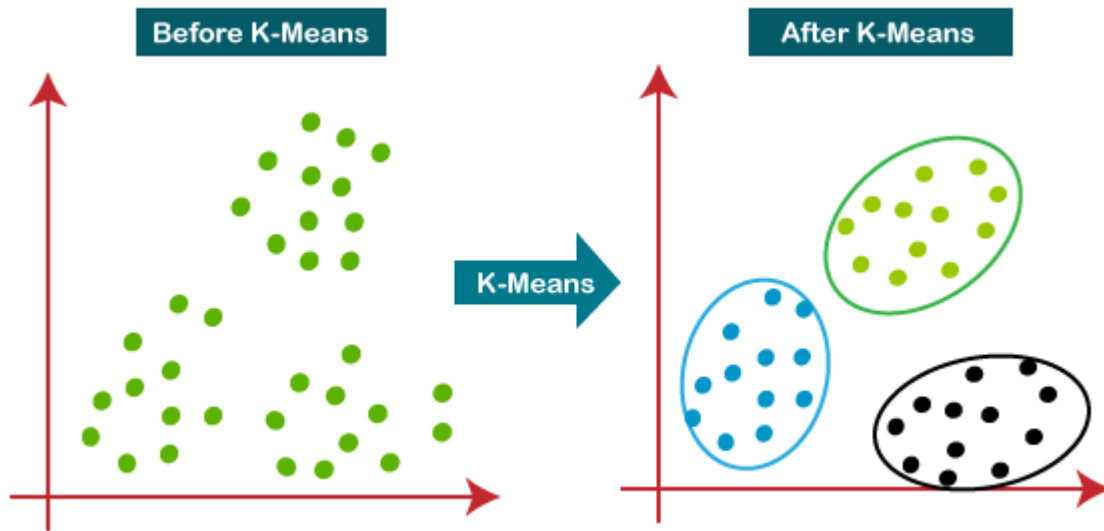
Distancia de Minkowski

$$d_p(x,y) = [\Sigma |x_i - y_i|^p]^{(1/p)}$$

Rango: $[0, \infty)$

Generalización: $p=1$ Manhattan, $p=2$ Euclidiana, $p \rightarrow \infty$ Chebyshev.

K-means



Es un método de clustering no supervisado cuyo objetivo es dividir un conjunto de datos en k grupos de tal forma que los elementos dentro de cada grupo sean lo más similares entre sí y lo más diferentes de los de otros grupos.

El algoritmo busca minimizar la suma de las distancias cuadradas entre cada punto y el centroide de su cluster, conocida como **Inercia**.

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Algoritmo

Algorithm 1 Pseudocode of K-Means.

Input: $X = \{x_1, x_2, \dots, x_n\}$ // dataset of n points in $\mathbb{R}^d$
 K // number of clusters
 ε // convergence tolerance
 max_iters // maximum number of iterations
Output: $M = \{m_1, \dots, m_K\}$ // final set of K centroids
 $L = \{\ell_1, \dots, \ell_n\}$ // cluster label for each point x_i
 J // final inertia (sum of squared distances)

1: $M \leftarrow \text{Random_Init}(X, K)$ // select K distinct points from X as initial centroids
2: $iter \leftarrow 0$

3: **while not** **Termination_Condition**(ε, max_iters) **do**

 // ——— STEP 1: Assignment ———

4: **for** $i \leftarrow 1$ **to** n **do**

5: $\ell_i \leftarrow \arg \min_k \|x_i - m_k\|_2$ // assign each point to nearest centroid

6: **end for**

 // ——— STEP 2: Update centroids ———

7: **for** $k \leftarrow 1$ **to** K **do**

8: **if** $|C_k| > 0$ **then**

9: $m_k \leftarrow \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ // recompute centroid as cluster mean

10: **else**

11: $m_k \leftarrow \text{Random_Reinit}(X)$ // reinitialize empty cluster centroid

12: **end if**

13: **end for**

 // ——— STEP 3: Compute inertia ———

14: $J \leftarrow \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - m_k\|_2^2$ // total sum of squared distances

 // ——— STEP 4: Convergence check ———

15: $\Delta \leftarrow \sum_{k=1}^K \|m_k^{(new)} - m_k^{(prev)}\|_2$ // total centroid displacement

16: **if** $\Delta < \varepsilon$ **then**

17: **stop** // centroids have stabilized

18: **end if**

19: $iter \leftarrow iter + 1$

20: **end while**

21: **return** M, L, J

1. Elegir el número de clusters k

Se define previamente cuantos clusters (clases) se desean. Es la única entrada que requiere criterio humano.

2. Inicializar centroides

Se seleccionan k centroides iniciales. Puede ser de forma aleatoria o con métodos más inteligentes como K-means++

```
 $M \leftarrow \text{Random\_Init}(X, K)$     // select  $K$  distinct points from  $X$  as initial centroids
```

3. Asignación

Para cada punto del dataset se calcula la distancia a cada centroide y se asigna al cluster con menor distancia.

$$\text{cluster}(x_i) = \arg \min_j ||x_i - \mu_j||$$

4. Actualizar los centroides

Se actualiza cada centroide como el promedio de los puntos asignados al cluster (se mueve el centroide al centro del cluster):

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

5. Repetir 3 y 4.

Hasta que: los centroides no cambian, las asignaciones no cambian o se alcanza el máximo de iteraciones

K-means

Algorithm 1 Pseudocode of K-Means.

Input: $X = \{x_1, x_2, \dots, x_n\}$ // dataset of n points in $\mathbb{R}^d$
 K // number of clusters
 ε // convergence tolerance
 max_iters // maximum number of iterations
Output: $M = \{m_1, \dots, m_K\}$ // final set of K centroids
 $L = \{\ell_1, \dots, \ell_n\}$ // cluster label for each point x_i
 J // final inertia (sum of squared distances)

1: $M \leftarrow \text{Random_Init}(X, K)$ // select K distinct points from X as initial centroids
2: $iter \leftarrow 0$

3: **while not** **Termination_Condition**(ε, max_iters) **do**

 // ——— STEP 1: Assignment ———

4: **for** $i \leftarrow 1$ **to** n **do**

5: $\ell_i \leftarrow \arg \min_k \|x_i - m_k\|_2$ // assign each point to nearest centroid

6: **end for**

 // ——— STEP 2: Update centroids ———

7: **for** $k \leftarrow 1$ **to** K **do**

8: **if** $|C_k| > 0$ **then**

9: $m_k \leftarrow \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ // recompute centroid as cluster mean

10: **else**

11: $m_k \leftarrow \text{Random_Reinit}(X)$ // reinitialize empty cluster centroid

12: **end if**

13: **end for**

 // ——— STEP 3: Compute inertia ———

14: $J \leftarrow \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - m_k\|_2^2$ // total sum of squared distances

 // ——— STEP 4: Convergence check ———

15: $\Delta \leftarrow \sum_{k=1}^K \|m_k^{(new)} - m_k^{(prev)}\|_2$ // total centroid displacement

16: **if** $\Delta < \varepsilon$ **then**

17: **stop** // centroids have stabilized

18: **end if**

19: $iter \leftarrow iter + 1$

20: **end while**

21: **return** M, L, J

En código

2. Inicializar centroides

```
def inicializar_centroides(X, k, metodo="kmeans++"):
    n = X.shape[0]

    if metodo == "aleatorio":
        indices = np.random.choice(n, k, replace=False)
        return X[indices].copy()

    elif metodo == "kmeans++":
        # Primer centroide: aleatorio
        centroides = [X[np.random.randint(n)]]

        for _ in range(k - 1):
            # Distancia de cada punto al centroide más cercano
            distancias = np.array([
                min(np.linalg.norm(x - c)**2 for c in centroides)
                for x in X
            ])
            # Elegir siguiente centroide con probabilidad proporcional a distancia²
            probs = distancias / distancias.sum()
            siguiente = X[np.random.choice(n, p=probs)]
            centroides.append(siguiente)

        return np.array(centroides)
```

3. Asignación

```
def asignar_clusters(X, centroides):  
    distancias = np.linalg.norm(X[:, np.newaxis] - centroides, axis=2)  
    return np.argmin(distancias, axis=1)
```

4. Actualización

```
def actualizar_centroides(X, etiquetas, k):  
    nuevos = np.zeros((k, X.shape[1]))  
    for j in range(k):  
        puntos = X[etiquetas == j]  
        if len(puntos) > 0:  
            nuevos[j] = puntos.mean(axis=0)  
    return nuevos
```

- K-means

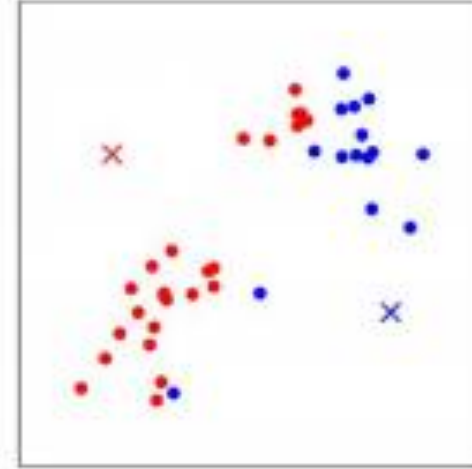
```
def kmeans(X, k, max_iter=100, tol=1e-4, metodo_init="kmeans++", seed=42):  
    np.random.seed(seed)  
    centroides = inicializar_centroides(X, k, metodo_init)  
    historial = []  
    inercias = []  
  
    for i in range(max_iter):  
        etiquetas = asignar_clusters(X, centroides)  
        nuevos_centroides = actualizar_centroides(X, etiquetas, k)  
        inercia = calcular_inercia(X, etiquetas, centroides)  
  
        historial.append((centroides.copy(), etiquetas.copy(), inercia))  
        inercias.append(inercia)  
  
        # Convergencia  
        if np.linalg.norm(nuevos_centroides - centroides) < tol:  
            print(f"Convergencia en iteración {i+1}")  
            centroides = nuevos_centroides  
            break  
  
        centroides = nuevos_centroides  
  
    etiquetas = asignar_clusters(X, centroides)  
    return etiquetas, centroides, historial, inercias
```



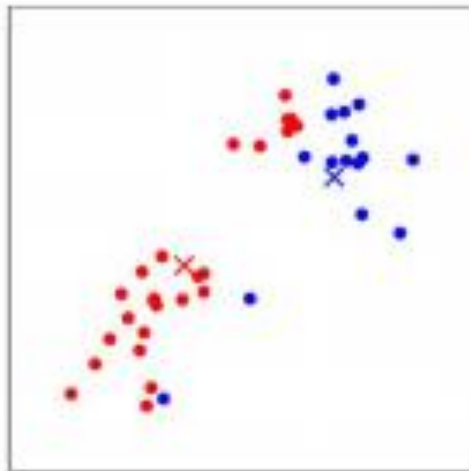
(a)



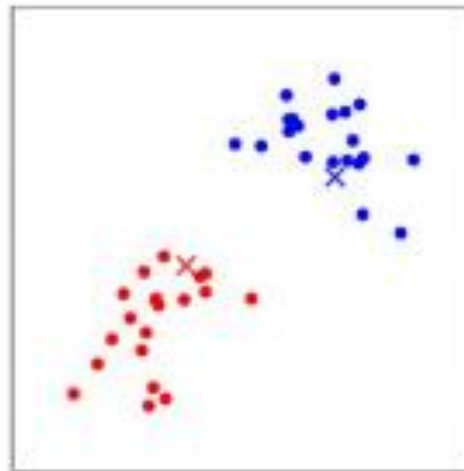
(b)



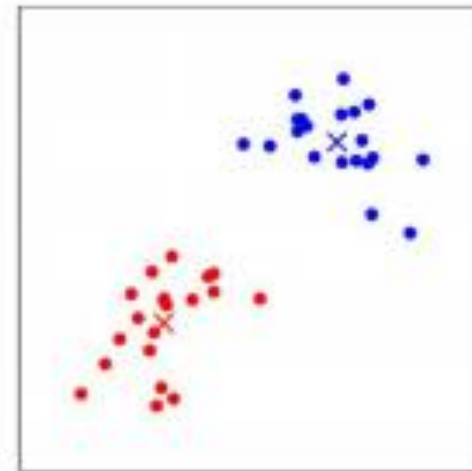
(c)



(d)



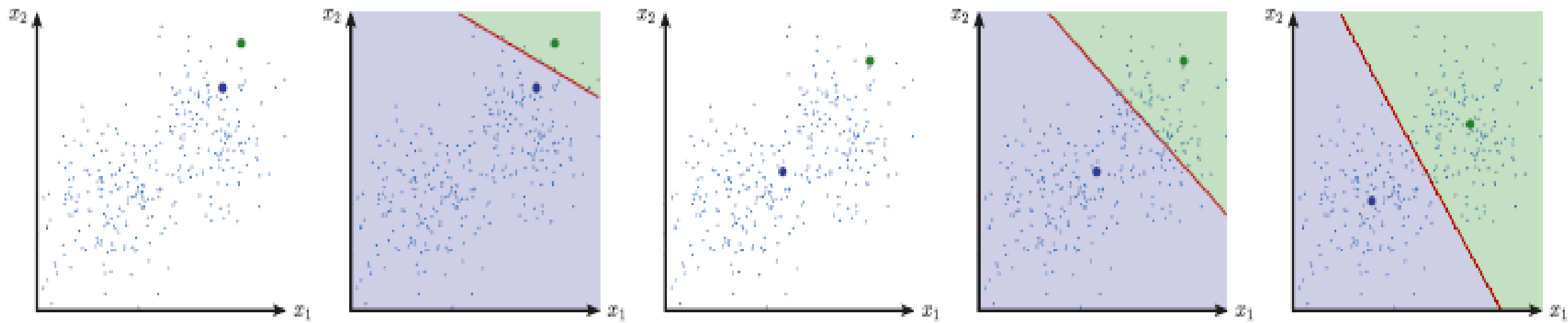
(e)



(f)

Consideraciones

Aspecto	Detalle
Complejidad	$O(n \cdot k \cdot i \cdot d)$ — n puntos, k clusters, i iteraciones, d dimensiones
Sensibilidad al inicio	Distintas inicializaciones pueden dar resultados diferentes
Limitaciones	Asume clusters esféricos Sensible a outliers y escala de los puntos Hay que elegir k previamente
Simplicidad	No es difícil de implementar
Escalable	Es posible implementarlo en datasets grandes



LBG (Linde-Buzo-Gay)

Es un método de clustering utilizado para diseñar codebooks en cuantización vectorial (Vector Quantization, VQ). Un codebook es un conjunto óptimo de centroides que represente un conjunto de datos.

A partir de un conjunto de entrenamiento X , se va a representar con k vectores prototipo llamados codewords. Cada vector de datos se aproxima por el codeword más cercano.

El algoritmo busca minimizar la distorsión:

$$D = \frac{1}{N} \sum_{i=1}^N \|x_i - c(x_i)\|^2$$

LBG

Algorithm 1 Pseudocode of LBG (Linde-Buzo-Gray).

Input: $X = \{x_1, x_2, \dots, x_n\}$ // dataset of n training vectors in $\mathbb{R}^d$
 N // target codebook size (must satisfy $N = 2^L$)
 ε // splitting perturbation factor
 δ // Lloyd convergence tolerance
 max_lloyd // maximum iterations for inner Lloyd loop

Output: $\mathcal{C} = \{c_1, \dots, c_N\}$ // final codebook of N codevectors
 $L = \{\ell_1, \dots, \ell_n\}$ // codevector index assigned to each x_i
 D // final average distortion

```

1:  $c_1 \leftarrow \frac{1}{n} \sum_{i=1}^n x_i$  // level 0: single codevector = global mean
2:  $\mathcal{C} \leftarrow \{c_1\}$  // codebook starts with one entry
3:  $K \leftarrow 1$  // current codebook size
4:  $level \leftarrow 0$ 

5: while  $K < N$  do
    // — STEP 1: Split every codevector —
6:    $\mathcal{C}_{new} \leftarrow \emptyset$ 
7:   for  $k \leftarrow 1$  to  $K$  do
8:      $c_k^+ \leftarrow c_k \cdot (1 + \varepsilon)$  // positive perturbation
9:      $c_k^- \leftarrow c_k \cdot (1 - \varepsilon)$  // negative perturbation
10:     $\mathcal{C}_{new} \leftarrow \mathcal{C}_{new} \cup \{c_k^+, c_k^-\}$ 
11:  end for
12:   $\mathcal{C} \leftarrow \mathcal{C}_{new}$ 
13:   $K \leftarrow 2K$  // codebook size doubles at each level
14:   $level \leftarrow level + 1$ 

```

```

    // — STEP 2: Refine with Lloyd algorithm —
15:   $D_{prev} \leftarrow \infty$ 
16:  for  $t \leftarrow 1$  to  $max\_lloyd$  do
    // 2a. Nearest-neighbor assignment
17:    for  $i \leftarrow 1$  to  $n$  do
18:       $\ell_i \leftarrow \arg \min_k \|x_i - c_k\|_2^2$  // assign  $x_i$  to closest codevector
19:    end for
    // 2b. Centroid update
20:    for  $k \leftarrow 1$  to  $K$  do
21:      if  $|V_k| > 0$  then
22:         $c_k \leftarrow \frac{1}{|V_k|} \sum_{x_i \in V_k} x_i$  //  $V_k$  = Voronoi cell of  $c_k$ 
23:      else
24:         $c_k \leftarrow c_k$  // keep codevector if cell is empty
25:      end if
26:    end for
    // 2c. Compute average distortion
27:     $D \leftarrow \frac{1}{n} \sum_{i=1}^n \|x_i - c_{\ell_i}\|_2^2$  // mean squared error over all training vectors
    // 2d. Check Lloyd convergence
28:    if  $\left| \frac{D_{prev} - D}{D_{prev}} \right| \leq \delta$  then
29:      break // relative distortion change below tolerance
30:    end if
31:     $D_{prev} \leftarrow D$ 
32:  end for
33:  print ( $level, K, D$ ) // report progress at each splitting level
34: end while

35: return  $\mathcal{C}, L, D$ 

```

1. Calcular el centroide global

Se parte de un solo centroide calculado como la media de todos los vectores del conjunto de entrenamiento:

$$c_1 = \frac{1}{N} \sum_{i=1}^N x_i$$

Este es el primer **codeword**.

2. Dividir los centroides (splitting)

Cada centroide se divide en dos agregando una pequeña perturbación ϵ :

$$c_i^+ = c_i(1 + \epsilon)$$

$$c_i^- = c_i(1 - \epsilon)$$

3. Asignación de vectores

Cada vector de entrenamiento x se asigna al centroide más cercano según una medida de distancia.

4. Actualización de centroides

Se recalcula cada centroide como la media de los vectores asignados a su región:

$$c_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

5. Calculo de la distorsión

Se calcula la distorsión promedio: $D = \frac{1}{N} \sum ||x_i - c(x_i)||^2$

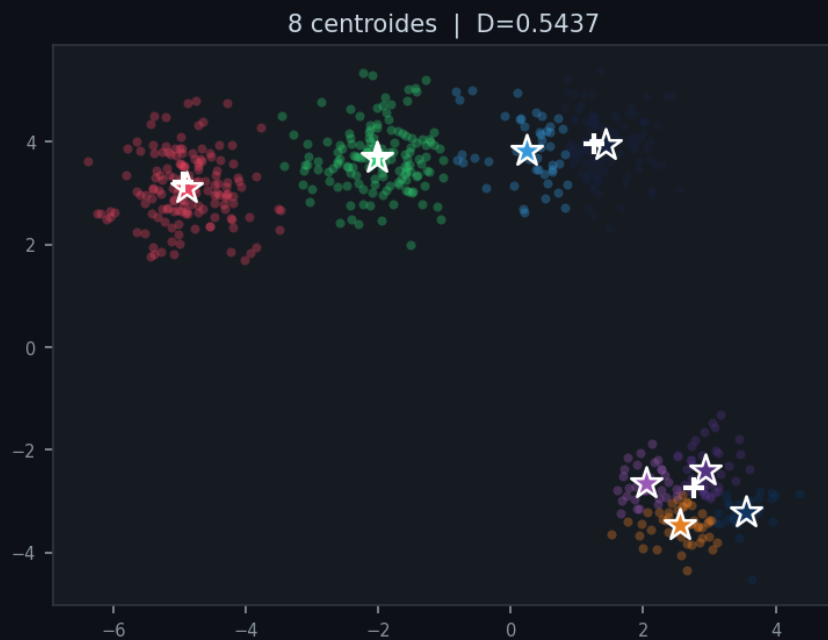
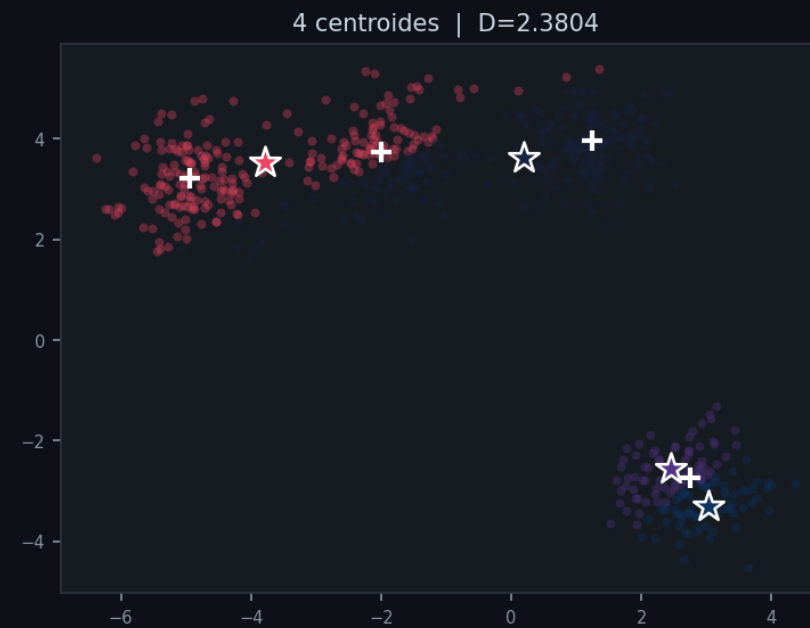
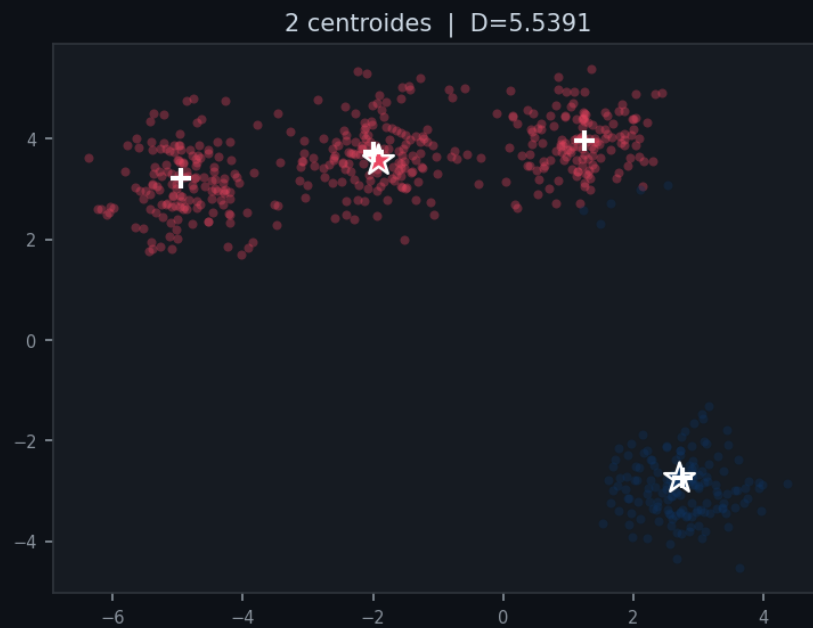
6. Convergencia

Si la mejora en la distorsión es pequeña: $\frac{D_{anterior} - D_{nuevo}}{D_{anterior}} < \delta$

entonces se considera que el algoritmo convergió para ese tamaño de codebook. De lo contrario se repiten los pasos 3 a 5.

7. Verificación del tamaño del codebook

Si el tamaño del codebook es menor al tamaño deseado volver al paso 2, de lo contrario el algoritmo termina



En código

1. Inicialización

```
# — Paso 1: centroide inicial (media global) —————  
codebook = np.mean(data, axis=0, keepdims=True) # shape (1, D)
```

2. Dividir centroides

```
# — Paso 2: Splitting —————  
c_plus = codebook * (1 + epsilon)  
c_minus = codebook * (1 - epsilon)  
codebook = np.vstack([c_plus, c_minus])
```

3. Asignación

```
# Paso 3: Asignación (nearest neighbor)  
distances = np.linalg.norm(data[:, np.newaxis, :] - codebook[np.newaxis, :, :], axis=2)  
labels = np.argmin(distances, axis=1)
```

4. Actualización

```
# Paso 4: Actualización de centroides
new_codebook = np.copy(codebook)
for j in range(len(codebook)):
    members = data[labels == j]
    if len(members) > 0:
        new_codebook[j] = members.mean(axis=0)

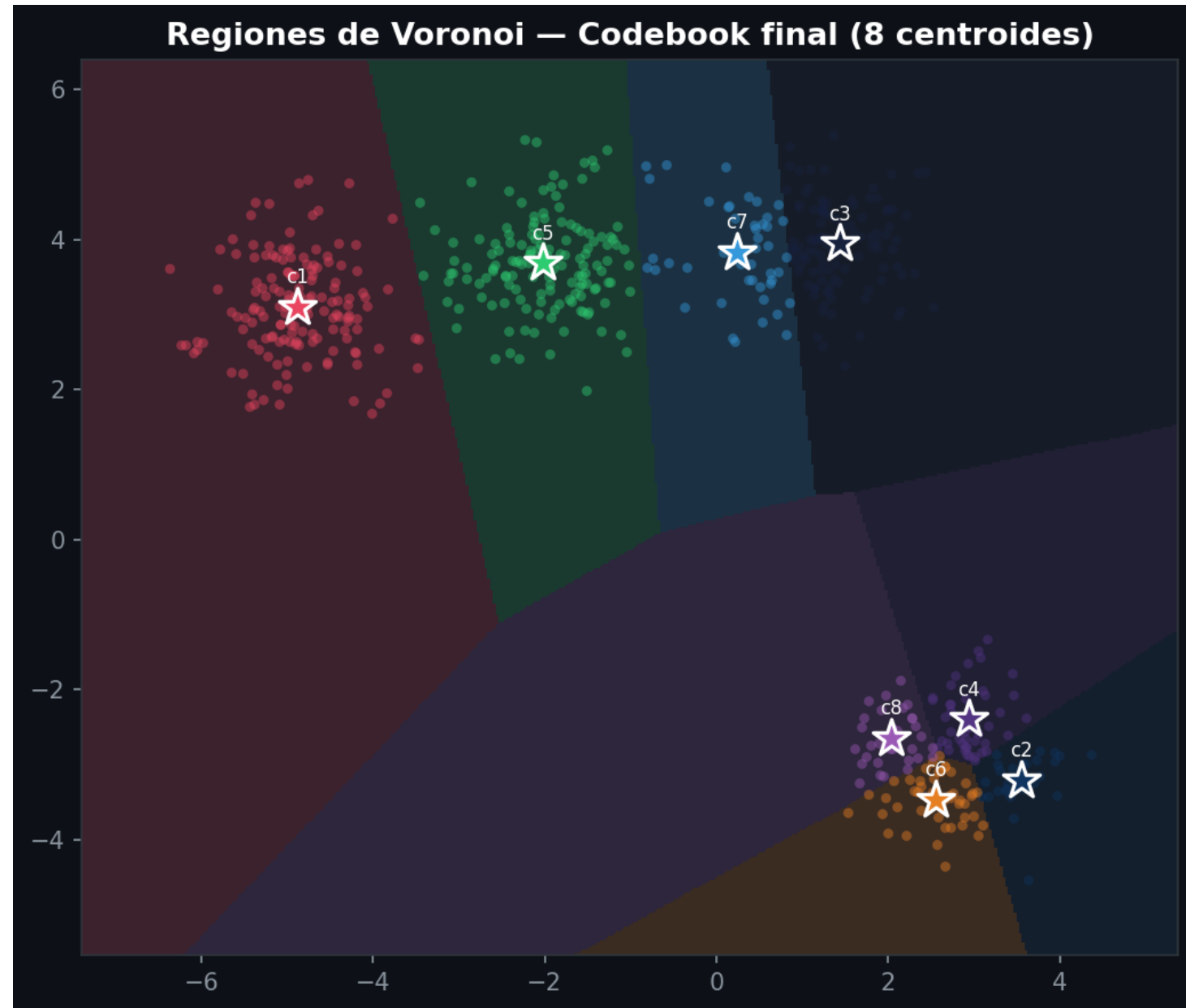
codebook = new_codebook
```

5. Calculo distorsión

```
# Paso 5: Distorsión
assigned = codebook[labels]
distortion = np.mean(np.sum((data - assigned) ** 2, axis=1))
```

6. Convergencia

```
# Paso 6: Convergencia
rel_change = abs(distortion_prev - distortion) / (distortion_prev + 1e-10)
if rel_change < convergence_threshold:
    break
distortion_prev = distortion
```



¿Cómo se usa en RP?

- Fase 1. Entrenamiento

Se construye un codebook para cada posible clase del conjunto de datos.

Clase 1 → codebook C1

Clase 2 → codebook C2

...

Clase C → codebook CC

- Fase 2. Clasificación

Dado un patrón nuevo x , se calcula la distancia al codeword más cercano de cada clase y se asigna a la clase con menor distancia.

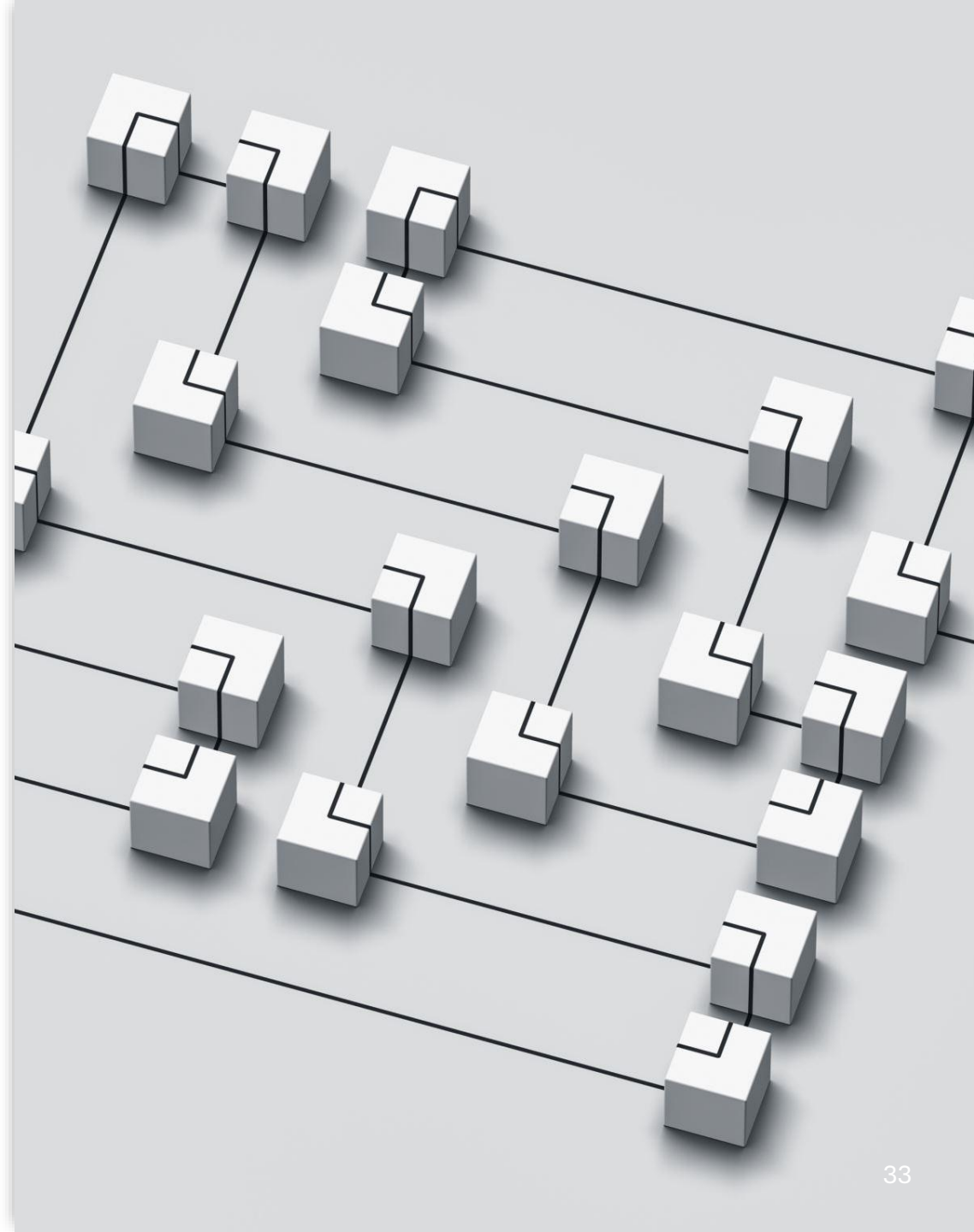
$$d(x, C_j) = \min_k \|x - c_{jk}\| \quad \hat{\omega} = \arg \min_j d(x, C_j)$$

Consideraciones

Aspecto	Detalle
Tamaño codebook	Muy pequeño: underfitting, Muy Grande: overfitting
Elección de ε	Demasiado pequeño los centroides tardan más en separarse; si es muy grande puede saltarse buenas configuraciones.
Métrica de distancia	La distancia euclídea es la más común, pero no siempre la más adecuada
Tamaño del conjunto	Si los datos de entrenamiento no cubren bien la distribución real, el codebook tendrá puntos ciegos
Limitaciones	Sensible a outliers. Queda atrapado en mínimos locales. N restringido a potencias de 2.
Ventajas	Representación compacta de los datos. Clasificación rápida. Método relativamente simple. Inicialización más estable que k-means.

Iterative Self-Organizing Data Analysis Technique Algorithm (ISODATA)

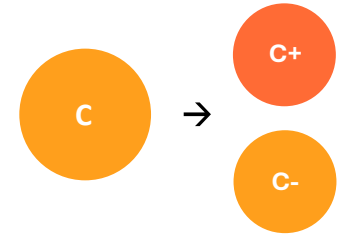
Es un algoritmo de clustering no supervisado similar a K-means, pero más flexible, ya que se distingue por su capacidad de ajustar automáticamente el número de clústeres durante la ejecución, en lugar de mantenerlo fijo.



Operaciones adicionales:

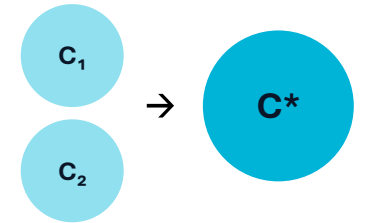
- División de clúster

Si la desviación estándar σ_k de un clúster supera un umbral $\sigma_{\max}$, se divide en dos subclusters.



- Fusión de clústeres

Si la distancia entre dos centroides m_i y m_j es menor que $d_{\min}$, ambos clústeres se fusionan.



- Eliminación de clústeres

Si el número de puntos en un clúster $|C_k| < N_{\min}$, el clúster se elimina y sus puntos se redistribuyen.

Parámetros del algoritmo

K_0 : Número inicial de clústeres

d_{min} : distancia mínima entre clústeres

N_{min} : Mínimo de puntos por clúster

θ_c : Máximo de fusiones por iteración

k_{max} : Limite superior de clústeres permitidos

σ_{max} : Desviación estándar máxima

Max_iters : Numero máximo de iteraciones

ISODATA

Algorithm 1 Pseudocode of ISODATA.

Input: $X = \{x_1, x_2, \dots, x_n\}$ // dataset of n points in $\mathbb{R}^d$
 K_{init} // initial number of clusters
 K_{max} // maximum number of clusters allowed
 N_{min} // minimum points per cluster (elimination threshold)
 σ_{max} // maximum std deviation per cluster (split threshold)
 d_{min} // minimum distance between centroids (merge threshold)
 θ_C // maximum merges per iteration
 ε, max_iters // convergence tolerance and iteration limit
Output: $M = \{m_1, \dots, m_{K_f}\}$ // final set of K_f centroids
 $L = \{\ell_1, \dots, \ell_n\}$ // cluster label for each point x_i

```

1:  $M \leftarrow \text{Random\_Init}(X, K_{init})$  // select  $K_{init}$  points from  $X$  as initial centroids

2: while not Termination_Condition( $\varepsilon, max\_iters$ ) do
    // — STEP 1: Assignment —
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $\ell_i \leftarrow \arg \min_k \|x_i - m_k\|_2$  // assign each point to nearest centroid
5:   end for
    // — STEP 2: Elimination —
6:   for  $k \leftarrow 1$  to  $K$  do
7:     if  $|C_k| < N_{min}$  then
8:       Remove( $m_k, M$ ) // discard cluster  $k$ ;  $K \leftarrow K - 1$ 
9:     end if
10:  end for
    // — STEP 3: Update centroids —
11:  for  $k \leftarrow 1$  to  $K$  do
12:     $m_k \leftarrow \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$  // recompute centroid as cluster mean
13:  end for

```

```

    // — STEP 4: Split —
14:  for  $k \leftarrow 1$  to  $K$  do
15:     $\sigma_k \leftarrow \max_j \sqrt{\frac{1}{|C_k|} \sum_{x_i \in C_k} (x_{ij} - m_{kj})^2}$  // max std dev across dimensions
16:    if  $\sigma_k > \sigma_{max}$  and  $K < K_{max}$  then
17:       $j^* \leftarrow \arg \max_j \sigma_{kj}$  // axis of greatest variance
18:       $\delta[j^*] \leftarrow \sigma_k \cdot 0.5$ ;  $\delta[j \neq j^*] \leftarrow 0$  // displacement vector
19:      Replace  $m_k$  with  $m_k^+ \leftarrow m_k + \delta$  and  $m_k^- \leftarrow m_k - \delta$ 
20:       $K \leftarrow K + 1$ 
21:    end if
22:  end for
    // — STEP 5: Merge —
23:   $merges \leftarrow 0$ 
24:  for all pairs  $(i, j)$  with  $i < j$  and  $merges < \theta_C$  do
25:    if  $\|m_i - m_j\|_2 < d_{min}$  then
26:       $m_{new} \leftarrow \frac{|C_i| \cdot m_i + |C_j| \cdot m_j}{|C_i| + |C_j|}$  // weighted average centroid
27:      Replace  $m_i$  with  $m_{new}$ ; Remove( $m_j, M$ )
28:       $K \leftarrow K - 1$ ;  $merges \leftarrow merges + 1$ 
29:    end if
30:  end for
    // — STEP 6: Convergence check —
31:   $\Delta \leftarrow \sum_k \|m_k^{(new)} - m_k^{(prev)}\|_2$ 
32:  if  $\Delta < \varepsilon$  then
33:    stop
34:  end if
35: end while

36: return  $M, L$ 

```

Pasos del Algoritmo ISODATA

1

Inicialización

Seleccionar k_0 centroides iniciales (aleatorio o heurístico).

2

Asignación

Asignar cada punto al centroide más cercano (distancia Euclidiana).

3

Actualización

Recalcular la media (centroide) de cada clúster con sus puntos.

4

Eliminación

Eliminar clústeres con menos de θ_n puntos.

5

División

Si $\sigma > \sigma_{max}$ y el clúster es grande $\rightarrow$ dividir en dos.

6

Fusión

Si distancia entre centroides $< d_{min} \rightarrow$ fusionar los clústeres.

7

¿Converge?

Si los cambios son mínimos o se alcanzó $max_iters \rightarrow$ FIN. Si no $\rightarrow$ volver a paso 2.

1. Inicialización

Se seleccionan K puntos del dataset como centroides iniciales. La forma más simple es selección aleatoria sin reemplazo.

centroides iniciales $\mu_1, \mu_2, \dots, \mu_K$

+

•

2. Asignación

Para cada punto x_i se calcula su distancia a todos los centroides y se asigna al más cercano.

○

$$c_i = \operatorname{argmin} \|x_i - \mu_j\|$$

3. Actualización

Para cada clúster C_i se recalcula el centroide como el promedio aritmético de todos sus puntos, dimensión por dimensión.

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x_i$$

4. Eliminación de clústeres pequeños

Se recorre cada clúster k y se cuenta cuántos puntos tiene. Si un clúster tiene $|C_j| < N_{min}$ entonces el clúster se elimina. Típicamente $N_{min} \leq n / k_{max}$

5. División de clústeres

Para cada clúster k se evalúa si su dispersión interna es demasiado alta. Se calcula la desviación estándar por dimensión:

$$\sigma_{j,k} = \sqrt{\frac{1}{|C_j|} \sum_{x_i \in C_j} (x_{i,k} - \mu_{j,k})^2}$$

Se calculan $\sigma_{k1}, \sigma_{k2}, \dots, \sigma_{kd}$ (una por cada dimensión) y se toma la máxima:

$$\sigma_k = \max_j \sigma_{kj}$$

$$j^* = \operatorname{argmax}_j \sigma_{kj}$$

Condición de división — se deben cumplir ambas subcondiciones simultáneamente:

Condición A: $\sigma_k > \sigma_{\max}$ (el cluster es demasiado disperso)

Condición B: $K_{\text{actual}} < K_{\max}$ (aún hay margen para más clusters)

Cálculo de los dos nuevos centroides si se cumplen A y B:

δ = vector de ceros en $\mathbb{R}^d$

$\delta[j^*] = \sigma_k \cdot 0.5$ (desplazamiento en el eje de mayor varianza)

$m_k^+ = m_k + \delta$ (centroide hijo 1)

$m_k^- = m_k - \delta$ (centroide hijo 2)



6. Fusión

Se evalúan todos los pares posibles de centroides para detectar cuáles están demasiado cerca:

$$d_{ij} = || \mu_i - \mu_j ||$$

Si $d(\mu_i, \mu_j) < d_{min} \rightarrow$ fusionar clusters i y j .

Los clústeres se fusionan:

$$\mu_{new} = \frac{n_i \mu_i + n_j \mu_j}{n_i + n_j}$$

Las fusiones se aplican en orden creciente de distancia. Se lleva un contador que se incrementa con cada fusión y si llega a θ_c no se realizan más fusiones en esa iteración.

7. Verificar convergencia

El algoritmo termina si:

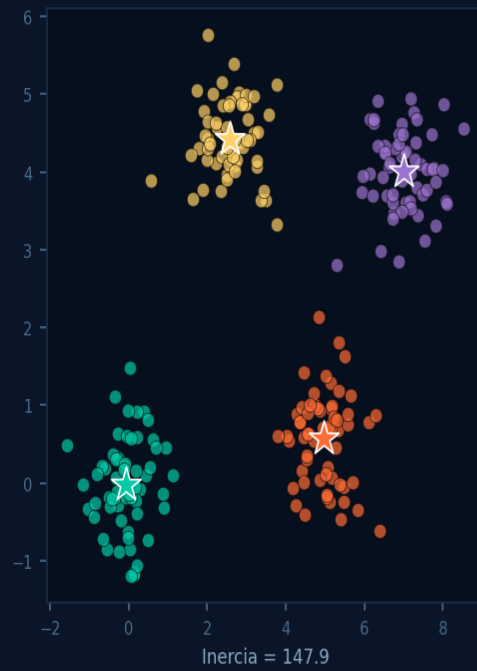
- Se alcanza el numero máximo de iteraciones
- Ningún punto cambio de clúster en esta iteración
- Se cumple:

$$\Delta < \varepsilon$$

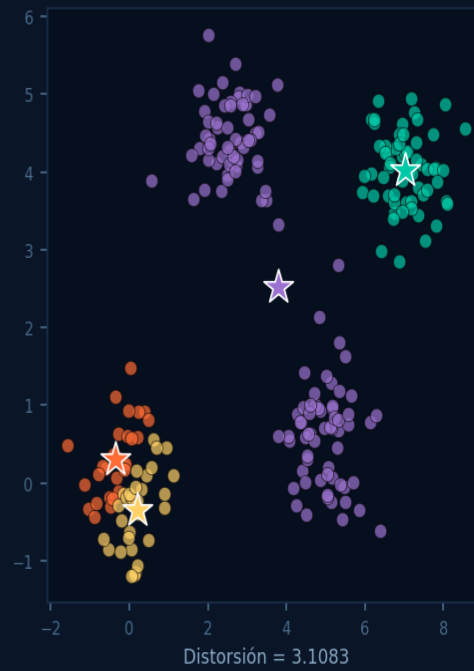
$$\Delta = \sum_k \left\| m_k^{(new)} - m_k^{(prev)} \right\|^2$$

Clasificación de Patrones · K-Medias · LBG · ISODATA

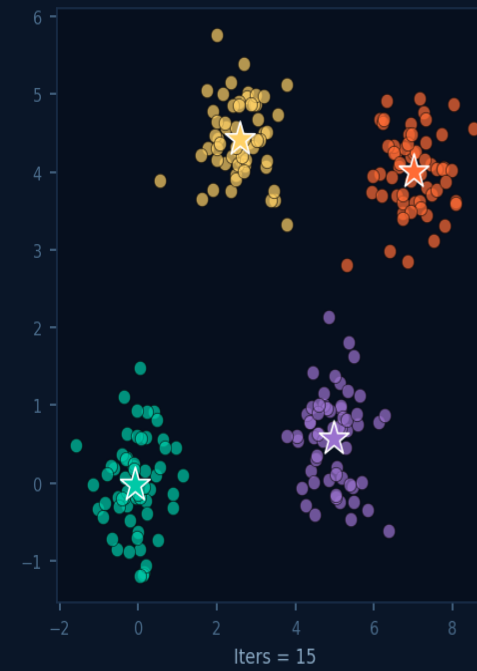
K-Medias (K=4)



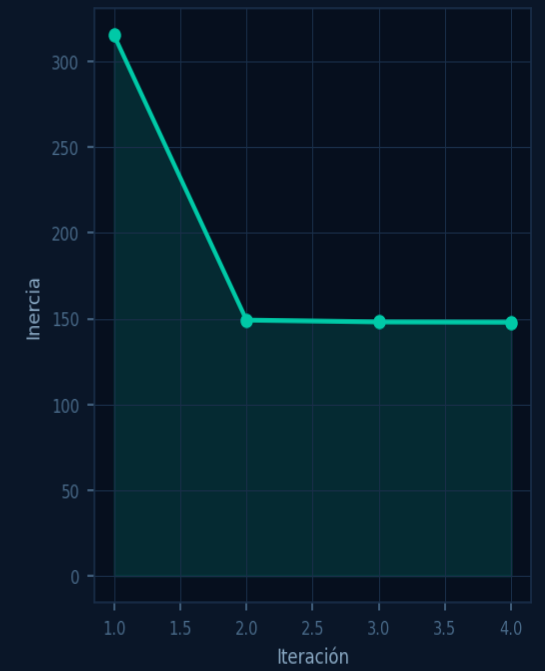
LBG (N=4)



ISODATA (K_final=4)

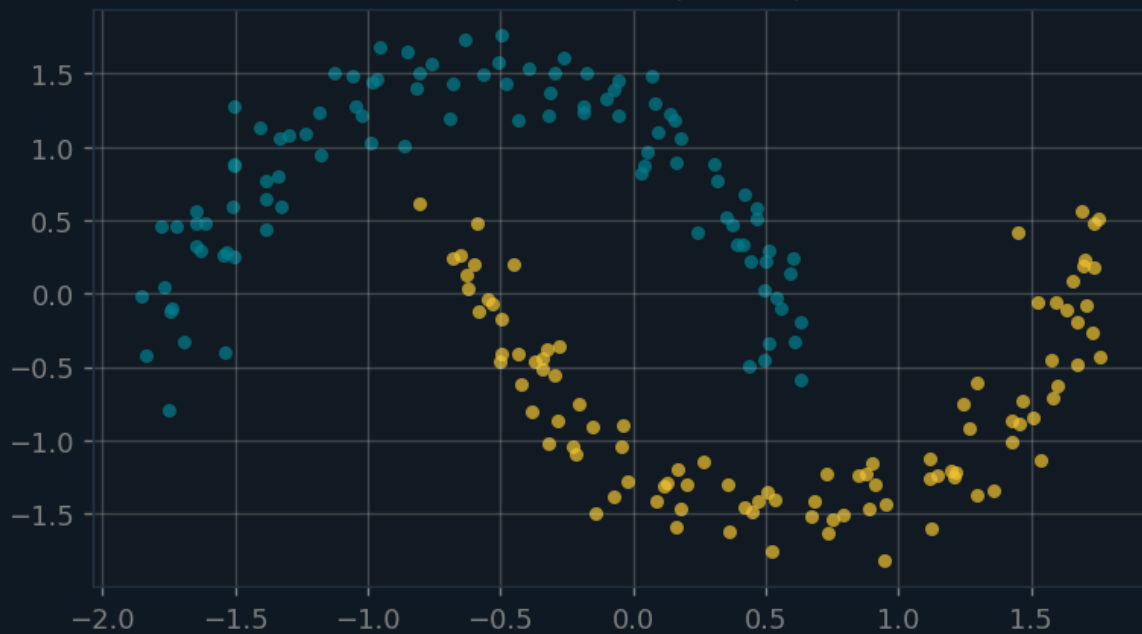


Convergencia K-Medias

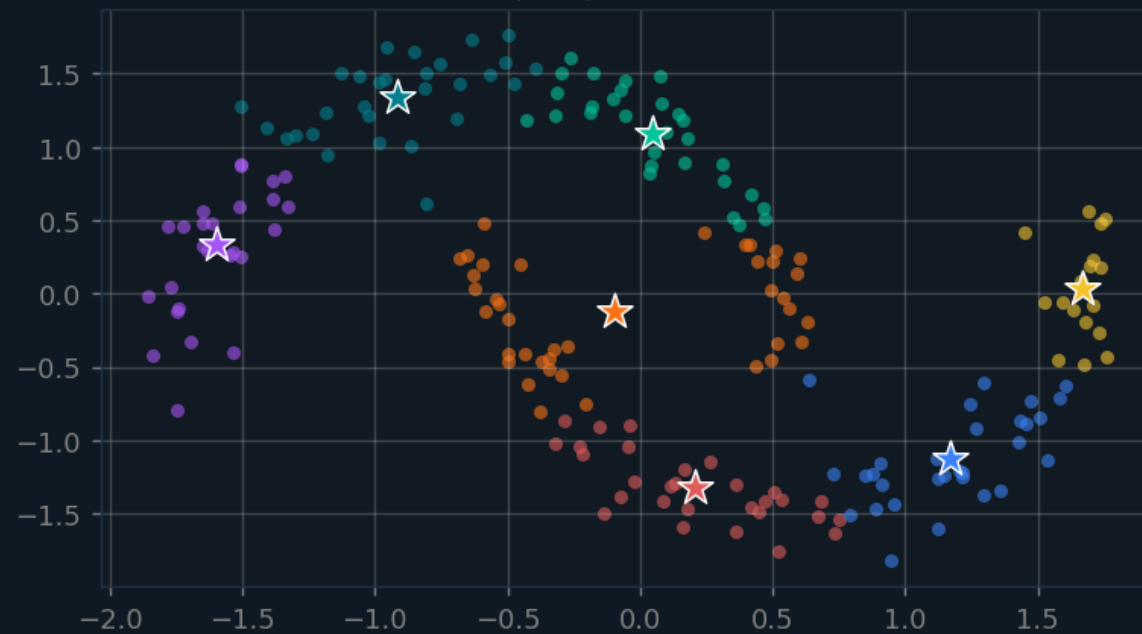


Limitación de ISODATA: datos no convexos

Estructura real (2 lunas)



ISODATA result (k=7) — formas no esféricas ⚠



Aplicaciones del Mundo Real



Teledetección

Clasificación de tipos de suelo y vegetación en imágenes satelitales (Landsat, Sentinel).



Medicina

Segmentación de imágenes médicas: tumores, tejidos en tomografías y resonancias.



Compresión de Datos

Reducción de la cantidad de colores en imágenes digitales (color quantization).



Minería de Datos

Descubrimiento de patrones en grandes bases de datos sin etiquetas previas.



SIG y Cartografía

Agrupación de regiones geográficas por similitud de características ambientales.



Visión por Computadora

Segmentación de escenas y detección de objetos en imágenes y video.

Consideraciones

Aspecto	Detalle
Inicialización	Los centroides iniciales influyen en el número final de clusters y estabilidad del algoritmo
Dependencia de parámetros heurísticos	distancia mínima desviación máxima tamaño mínimo del cluster
Forma de clusters	clusters aproximadamente esféricos
Ventajas	Número de clusters adaptable Maneja clusters con diferente tamaño Detecta clusters con alta varianza
Desventajas	Muchos parámetros Heurístico Sensible a ruido No garantiza solución optima

¿Cómo elegir los parámetros?

Parámetro	Definición
Distancia mínima de fusión D_{min}	$D_{min} \approx 0.5 \times \text{distancia promedio entre centroides}$ $D_{min} = \alpha \cdot \sigma_{global}$
Desviación estándar máxima S_{max}	$S_{max} \approx 1.5 - 2 \times \sigma_{global}$
Número mínimo de puntos N_{min}	$N_{min} \approx 1\% - 5\% \text{ del dataset}$ $N_{min} = N/10K$

Comparativa de Algoritmos

Criterio	K-Medias	LBG	ISODATA
Cuando usarlo	Se conoce k y los datos son simples	Se necesita un codebook de tamaño potencia de 2 con inicialización robusta	No se conoce k exacto y los datos son complejos o heterogéneos
K inicial	Requerido y fijo	K=1, crece x2	Estimado, varía
Inicialización	Aleatoria	Divisiva (global→local)	Manual / aleatoria
Clusters	Fijos durante proceso	Fijos al final	División y fusión dinámica
Criterio parada	Centroides estables	Distorsión $< \epsilon$	Múltiples umbrales
Sensibilidad a la inicialización	Alta	Baja	Media
Aplicación típica	Segmentación general	Compresión de audio/imagen	Imágenes satélite, señales

Resumen

Distancia Mínima

Clasificador que asigna cada patrón a la clase cuyo prototipo es más cercano. Requiere funciones de distancia (Euclidiana, Manhattan, Mahalanobis).

Medidas de Similitud

Cuantifican la afinidad entre vectores: similitud coseno, correlación de Pearson, Jaccard, Minkowski. Base matemática de todo algoritmo de agrupamiento.

Algoritmo K-Means

K-Medias

Minimiza la inercia intra-cluster. Iterativo, eficiente, sensible a inicialización. Ampliamente usado para segmentación.

Linde-Buzo-Gray

LBG

Enfocado en cuantización vectorial. Estrategia divisiva jerárquica (splitting). Genera codebooks de tamaño 2^n con mínima distorsión.

ISODATA

ISO

Extensión dinámica de K-medias. Permite dividir clusters elongados y fusionar clusters cercanos. Adapta K automáticamente.

Para la próxima clase...

Clasificación por funciones de
similitud

